Module 7) Python – Collections, functions and Modules

6. Generators and Iterators: (from module-6)

Q.1. Understanding how generators work in Python.

Ans: A **generator** is a special type of function that produces values **one at a time instead of all at once**. This helps save memory and makes programs efficient.

Generators use the **yield keyword** instead of return. When yield is used, the function **pauses** and resumes from the same point when called again.

Example

```
def numbers():
  yield 1
  yield 2
  yield 3

for num in numbers():
  print(num)
```

Output:

```
1
2
3
```

Here the generator returns values one by one.

Another Example

```
def count(n):
  i = 1
  while i <= n:
    yield i
    i += 1

for val in count(4):
  print(val)
```

Output:

```
1
2
```

3
4

Conclusion

Generators use **yield to produce values step by step** instead of returning all values at once. This makes them useful for handling large data efficiently.

Q.2. Difference between yield and return.

Ans: Both **return** and **yield** are used inside functions, but they behave differently.

1. return

- It **returns a value and ends the function**.
- Once return is executed, the function stops completely.

Example:

```
def add(a, b):
  return a + b

print(add(2, 3))
```

Output:

5

2. yield

- It **returns a value but pauses the function**.
- The function can continue from the same point when called again.
- It is used in **generator functions**.

Example:

```
def numbers():
  yield 1
  yield 2
  yield 3

for n in numbers():
  print(n)
```

Output:

1
2
3

Key Difference

- **return → ends the function completely**
- **yield → pauses the function and continues later**

Conclusion

return is used when we need a single result and want to stop the function, while yield is used to generate values **one by one** without ending the function.

Q.3. Understanding iterators and creating custom iterators

Ans: An **iterator** in Python is an object that allows us to **access elements one by one** from a collection like a list or tuple.

Iterators use two methods:

- __iter__() → returns the iterator object
- __next__() → returns the next value

Example of Iterator

```
numbers = [1, 2, 3]

it = iter(numbers)

print(next(it))
print(next(it))
print(next(it))
```

Output:

1
2
3

Here iter() creates an iterator and next() gives the next element.

Creating a Custom Iterator

We can create our own iterator using a class.

Example:

```
class Count:
  def __init__(self, n):
    self.n = n
    self.i = 1

  def __iter__(self):
    return self

  def __next__(self):
    if self.i <= self.n:
      value = self.i
      self.i += 1
      return value
    else:
      raise StopIteration

obj = Count(3)

for num in obj:
  print(num)
```

Output:

```
1
2
3
```

Conclusion

Iterators help us **process elements one at a time**. We can use built-in iterators or create **custom iterators** using __iter__() and __next__() methods.

7. Functions and Methods:

Q.1. Defining and calling functions in Python.

Ans: A **function** is a block of code that performs a specific task. It helps us **reuse code** and make programs organized.

1. Defining a Function

We define a function using the **def keyword**.

Syntax:

```
def function_name():
  # code
```

Example:

```
def greet():
  print("Hello, welcome")
```

2. Calling a Function

After defining, we call the function by writing its name with parentheses.

Example:

```
greet()
```

Output:

Hello, welcome

Example with Parameters

```
def add(a, b):
  print(a + b)

add(5, 3)
```

Output:

8

Conclusion

Functions are defined using def and executed by calling them. They help us write **clean, reusable, and organized code**.

Q.2. Function arguments (positional, keyword, default).

Ans: Function arguments are values passed to a function when calling it. Python supports different types of arguments.

1. Positional Arguments

In **positional arguments**, values are passed in the **same order** as parameters.

Example:

```
def greet(name, age):
  print(name, age)

greet("Mithlesh", 25)
```

Output:

Mithlesh 25

2. Keyword Arguments

In **keyword arguments**, we specify the parameter name while passing values. Order does not matter.

Example:

```
def greet(name, age):
  print(name, age)

greet(age=25, name="Mithlesh")
```

Output:

Mithlesh 25

3. Default Arguments

In **default arguments**, we provide a default value to parameters. If no value is passed, the default is used.

Example:

```
def greet(name, age=20):
  print(name, age)

greet("Mithlesh")
```

Output:

Mithlesh 20

Conclusion

Python supports **positional, keyword, and default arguments** to make functions flexible and easy to use.

Q.3. Scope of variables in Python.

Ans: The **scope of a variable** means the area of a program where the variable can be accessed. In Python, variables have different scopes depending on where they are defined.

1. Local Scope

A variable defined inside a function is called a **local variable**. It can be used only inside that function.

Example:

```
def func():
  x = 10
  print(x)

func()
```

Output:

10

Here x is only accessible inside func().

2. Global Scope

A variable defined outside all functions is called a **global variable**. It can be accessed anywhere in the program.

Example:

```
x = 20

def func():
  print(x)

func()
```

Output:

20

3. Using Global Keyword

If we want to modify a global variable inside a function, we use the **global keyword**.

Example:

```
x = 5
```

```
def func():
  global x
  x = 10

func()
print(x)
```

Output:

10

Conclusion

Variables in Python have **local and global scope**. Local variables are used inside functions, while global variables can be accessed throughout the program.

Q.4. Built-in methods forstrings, lists, etc.

Ans: Python provides many **built-in methods** to work easily with data types like strings and lists. These methods help us perform common operations quickly.

1. String Methods

String methods are used to **modify or analyze text**.

Example:

```
text = "python"

print(text.upper())
print(text.lower())
print(text.capitalize())
```

Output:

```
PYTHON
python
Python
```

2. List Methods

List methods help us **add, remove, or modify elements**.

Example:

```
numbers = [1, 2, 3]

numbers.append(4)
```

```
numbers.remove(2)
print(numbers)
```

Output:

[1, 3, 4]

3. Other Useful Methods

Example with len():

```
name = "Python"
print(len(name))
```

Output:

6

Example with type():

```
a = 10
print(type(a))
```

Output:

<class 'int'>

Conclusion

Built-in methods make Python powerful and easy to use. They help us **process strings, manage lists, and perform common tasks efficiently** without writing extra code.

10. Advanced Python (map(), reduce(), filter(), Closures and Decorators):

Q.1. How functional programming works in Python

Ans: Functional programming is a style of programming where we use **functions as the main building blocks**. In this approach, we focus on writing **small, reusable functions** instead of changing data directly.

Key Idea

In functional programming, we try to:

- Use functions to perform tasks
- Avoid changing data (immutability)
- Write clean and simple code

1. Using Functions as Values

In Python, functions can be stored in variables and passed as arguments.

Example:

```
def greet(name):
  return "Hello " + name

f = greet
print(f("Mithlesh"))
```

Output:

Hello Mithlesh

2. Using map()

The **map()** function applies a function to all elements in a list.

Example:

```
numbers = [1, 2, 3]
result = list(map(lambda x: x * 2, numbers))
print(result)
```

Output:

[2, 4, 6]

3. Using filter()

The **filter()** function selects elements based on a condition.

Example:

```
numbers = [1, 2, 3, 4]
result = list(filter(lambda x: x % 2 == 0, numbers))
print(result)
```

Output:

[2, 4]

Conclusion

Functional programming in Python focuses on using **functions, avoiding unnecessary changes, and writing clean code**. Functions like **map() and filter()** help us process data in a simple and efficient way.

Q.2. Using map(), reduce(), and filter() functions for processing data.

Ans: In Python, **map(), filter(), and reduce()** are used in functional programming to process data efficiently.

1. map() Function

The **map()** function applies a function to each element of a list.

Example:

```
numbers = [1, 2, 3]

result = list(map(lambda x: x * 2, numbers))
print(result)
```

Output:

[2, 4, 6]

2. filter() Function

The **filter()** function selects elements based on a condition.

Example:

```
numbers = [1, 2, 3, 4, 5]

result = list(filter(lambda x: x % 2 == 0, numbers))
print(result)
```

Output:

[2, 4]

3. reduce() Function

The **reduce()** function applies a function cumulatively to reduce the list to a single value. It is available in the **functools module**.

Example:

```
from functools import reduce

numbers = [1, 2, 3, 4]

result = reduce(lambda x, y: x + y, numbers)
print(result)
```

Output:

Conclusion

- **map()** → applies a function to all elements
- **filter()** → selects elements based on condition
- **reduce()** → reduces elements to a single value

These functions help us process data in a simple and efficient way.

Q.3. Introduction to closures and decorators.

Ans: Closures and decorators are advanced concepts in Python that help us write **clean and reusable code using functions**.

1. Closures

A **closure** is a function inside another function that **remembers the outer function's variables**, even after the outer function has finished.

Example:

```
def outer(x):
  def inner():
    print(x)
  return inner

func = outer(10)
func()
```

Output:

10

Here, the inner function remembers the value of x.

2. Decorators

A **decorator** is a function that is used to **modify or extend the behavior of another function** without changing its code.

Example:

```
def decorator(func):
  def wrapper():
    print("Before function")
    func()
```

```
        print("After function")
    return wrapper

@decorator
def greet():
    print("Hello")

greet()
```

Output:

```
Before function
Hello
After function
```

Conclusion

Closures allow a function to **remember values**, while decorators help us **add extra functionality to functions**. Both concepts are useful for writing clean and efficient Python code.

==========

1. Accessing List:

Q.1. Understanding how to create and access elements in a list.

Ans: A **list** in Python is a collection of items stored in **square brackets []**. Lists can store different types of data and can be changed (mutable).

1. Creating a List

We create a list by placing elements inside square brackets, separated by commas.

Example:

```
numbers = [10, 20, 30, 40]
print(numbers)
```

Output:

```
[10, 20, 30, 40]
```

2. Accessing Elements in a List

We can access elements using **index numbers**, starting from **0**.

Example:

```
fruits = ["apple", "banana", "mango"]

print(fruits[0])
print(fruits[2])
```

Output:

```
apple
mango
```

Here fruits[0] gives the first element and fruits[2] gives the third element.

3. Accessing Using Negative Index

We can also access elements from the end using negative indexing.

Example:

```
print(fruits[-1])
```

Output:

```
mango
```

Conclusion

Lists are used to store multiple items in a single variable. We can create lists using [] and access elements using **indexing (positive or negative)**, which makes it easy to work with collections of data.

Q.2. Indexing in lists (positive and negative indexing).

Ans: Indexing in Python lists is used to **access elements using their position**. There are two types of indexing: **positive indexing** and **negative indexing**.

1. Positive Indexing

In positive indexing, counting starts from **0 (left to right)**.

Example:

```
fruits = ["apple", "banana", "mango"]

print(fruits[0])
print(fruits[1])
```

Output:

apple
banana

Here fruits[0] is the first element and fruits[1] is the second element.

2. Negative Indexing

In negative indexing, counting starts from **-1 (right to left)**.

Example:

```
fruits = ["apple", "banana", "mango"]

print(fruits[-1])
print(fruits[-2])
```

Output:

mango
banana

Here fruits[-1] gives the last element and fruits[-2] gives the second last element.

Conclusion

Positive indexing starts from the beginning of the list, while negative indexing starts from the end. Both methods help us easily access elements in a list.

Q.3. Slicing a list: accessing a range of elements.

Ans: **List slicing** is used to get a **part (range) of elements** from a list. It allows us to select multiple elements at once.

The basic syntax is:

```
list[start:end]
```

Here, **start** is the starting index and **end** is the ending index (not included).

Example

```
numbers = [10, 20, 30, 40, 50]

print(numbers[1:4])
```

Output:

[20, 30, 40]

This selects elements from index **1 to 3**.

Example without Start or End

```
print(numbers[:3])   # from beginning
print(numbers[2:])   # till end
```

Output:

```
[10, 20, 30]
[30, 40, 50]
```

Example with Step

```
print(numbers[0:5:2])
```

Output:

```
[10, 30, 50]
```

This selects elements by skipping every second element.

Conclusion

List slicing helps us **access a range of elements easily** using start, end, and step values. It makes working with lists more flexible and efficient.

2. list operations:

Q.1. Common list operations: concatenation, repetition, membership.

Ans: In Python, lists support different operations that help us work with data easily. Some common operations are **concatenation, repetition, and membership**.

1. Concatenation

Concatenation means **joining two lists together** using the + operator.

Example:

```
list1 = [1, 2]
list2 = [3, 4]

result = list1 + list2
print(result)
```

Output:

[1, 2, 3, 4]

2. Repetition

Repetition means **repeating a list multiple times** using the * operator.

Example:

```
numbers = [1, 2]

print(numbers * 3)
```

Output:

[1, 2, 1, 2, 1, 2]

3. Membership

Membership is used to check whether an element exists in a list using **in** or **not in**.

Example:

```
fruits = ["apple", "banana", "mango"]

print("apple" in fruits)
print("grapes" not in fruits)
```

Output:

True
True

Conclusion

List operations like **concatenation (+), repetition (*), and membership (in, not in)** help us combine lists, repeat elements, and check the presence of values easily.

Q.2. Understanding list methods like append(), insert(), remove(), pop().

Ans: The **remove()** method is used to **delete a specific value** from the list.

Example:

```
numbers = [1, 2, 3, 2]
numbers.remove(2)
print(numbers)
```

Output:

[1, 3, 2]

It removes the **first occurrence** of the value.

4. pop()

The **pop()** method is used to **remove an element by index**. If no index is given, it removes the last element.

Example:

```
numbers = [1, 2, 3]
numbers.pop()
print(numbers)
```

Output:

[1, 2]

Another example:

```
numbers = [1, 2, 3]
numbers.pop(1)
print(numbers)
```

Output:

[1, 3]

Conclusion

List methods like **append(), insert(), remove(), and pop()** help us easily **add and remove elements** from a list, making list handling simple and flexible.

3. Working with Lists:

Q.1. Iterating over a list using loops.

Ans: Iterating over a list means **going through each element one by one**. We use loops to access and work with every item in the list.

1. Using for Loop

The **for loop** is the most common way to iterate over a list.

Example:

```
fruits = ["apple", "banana", "mango"]

for fruit in fruits:
  print(fruit)
```

Output:

apple
banana
mango

Here the loop accesses each element of the list one by one.

2. Using for Loop with Index

We can also use index values with the loop.

Example:

```
numbers = [10, 20, 30]

for i in range(len(numbers)):
  print(numbers[i])
```

Output:

10
20
30

3. Using while Loop

We can iterate using a **while loop** as well.

Example:

```
numbers = [10, 20, 30]
i = 0

while i < len(numbers):
  print(numbers[i])
  i = i + 1
```

Output:

10
20
30

Conclusion

We can iterate over a list using **for loops or while loops**. This helps us access and process each element of the list easily.

Q.2. Sorting and reversing a list using sort(), sorted(), and reverse().

Ans: In Python, we can arrange list elements in order or reverse them using built-in methods.

1. sort()

The **sort()** method is used to **sort the list in ascending order**. It changes the original list.

Example:

```
numbers = [4, 2, 1, 3]
numbers.sort()
print(numbers)
```

Output:

[1, 2, 3, 4]

2. sorted()

The **sorted()** function is used to **return a new sorted list** without changing the original list.

Example:

```
numbers = [4, 2, 1, 3]
new_list = sorted(numbers)
print(new_list)
```

Output:

[1, 2, 3, 4]

3. reverse()

The **reverse()** method is used to **reverse the order of elements** in the list.

Example:

```
numbers = [1, 2, 3, 4]
numbers.reverse()
print(numbers)
```

Output:

[4, 3, 2, 1]

Conclusion

- **sort()** sorts the original list
- **sorted()** returns a new sorted list
- **reverse()** reverses the list order

These methods help us easily organize and manipulate list data in Python.

Q.3. Basic list manipulations: addition, deletion, updating, and slicing.

Ans: In Python, we can easily modify lists by adding, removing, updating elements, and extracting parts of the list.

1. Addition (Adding Elements)

We can add elements using append() or insert().

Example:

```
numbers = [1, 2, 3]
numbers.append(4)
print(numbers)
```

Output:

[1, 2, 3, 4]

2. Deletion (Removing Elements)

We can remove elements using remove() or pop().

Example:

```
numbers = [1, 2, 3]
numbers.remove(2)
print(numbers)
```

Output:

[1, 3]

3. Updating Elements

We can change values by using index positions.

Example:

```
numbers = [1, 2, 3]
numbers[1] = 10
print(numbers)
```

Output:

[1, 10, 3]

4. Slicing (Accessing Part of List)

We can get a range of elements using slicing.

Example:

```
numbers = [1, 2, 3, 4, 5]
print(numbers[1:4])
```

Output:

[2, 3, 4]

Conclusion

List manipulation includes **adding, deleting, updating, and slicing elements**. These operations help us manage and modify list data easily in Python programs.

4. Tuple:

Q.1. Introduction to tuples, immutability.

Ans: A **tuple** in Python is a collection of elements stored in **parentheses ()**. It is similar to a list, but the main difference is that tuples are **immutable**, which means their values cannot be changed after creation.

Creating a Tuple

Example:

```
numbers = (1, 2, 3)
print(numbers)
```

Output:

(1, 2, 3)

Accessing Elements

We can access tuple elements using indexing, just like lists.

Example:

```
fruits = ("apple", "banana", "mango")
print(fruits[1])
```

Output:

banana

Immutability of Tuples

Once a tuple is created, we **cannot modify, add, or remove elements**.

Example:

```
numbers = (1, 2, 3)
numbers[0] = 10   # This will give an error
```

This shows that tuples do not allow changes after creation.

Conclusion

Tuples are used to store multiple values like lists, but they are **immutable**. This makes them useful when we want to keep data fixed and secure from changes.

Q.2. Creating and accessing elements in a tuple.

Ans: A **tuple** is a collection of elements stored in **parentheses ()**. Tuples can hold different types of data and are **immutable** (cannot be changed).

1. Creating a Tuple

We create a tuple by placing elements inside parentheses.

Example:

```
numbers = (10, 20, 30)
print(numbers)
```

Output:

(10, 20, 30)

We can also create a single-element tuple:

```
single = (5,)
print(single)
```

2. Accessing Elements

We can access elements using **indexing**, starting from **0**.

Example:

```
fruits = ("apple", "banana", "mango")

print(fruits[0])
print(fruits[2])
```

Output:

```
apple
mango
```

3. Negative Indexing

We can also access elements from the end using negative indexing.

Example:

```
print(fruits[-1])
```

Output:

mango

Conclusion

Tuples are created using () and we can access their elements using **positive and negative indexing**. They are useful when we want to store fixed data.

Q.3. Basic operations with tuples: concatenation, repetition, membership.

Ans: Tuples in Python support several basic operations that help us work with their elements easily.

1. Concatenation

Concatenation means joining two tuples using the + operator.

Example:

```
t1 = (1, 2)
t2 = (3, 4)

result = t1 + t2
print(result)
```

Output:

(1, 2, 3, 4)

2. Repetition

Repetition means repeating a tuple multiple times using the * operator.

Example:

```
t = (1, 2)

print(t * 3)
```

Output:

(1, 2, 1, 2, 1, 2)

3. Membership

Membership is used to check if an element exists in a tuple using in or not in.

Example:

```
t = (10, 20, 30)

print(20 in t)
print(40 not in t)
```

Output:

```
True
True
```

Conclusion

Tuple operations like **concatenation (+), repetition (*), and membership (in, not in)** help us combine tuples, repeat elements, and check for the presence of values easily.

5. Accessing Tuples:

Q.1. Accessing tuple elements using positive and negative indexing.

Ans: In Python, we can access elements of a tuple using **indexing**. There are two types: **positive indexing** and **negative indexing**.

1. Positive Indexing

In positive indexing, counting starts from **0 (left to right)**.

Example:

```
fruits = ("apple", "banana", "mango")

print(fruits[0])
print(fruits[1])
```

Output:

```
apple
banana
```

Here fruits[0] is the first element and fruits[1] is the second element.

2. Negative Indexing

In negative indexing, counting starts from **-1 (right to left)**.

Example:

```
fruits = ("apple", "banana", "mango")

print(fruits[-1])
print(fruits[-2])
```

Output:

```
mango
banana
```

Here fruits[-1] gives the last element and fruits[-2] gives the second last element.

Conclusion

Positive indexing starts from the beginning of the tuple, while negative indexing starts from the end. Both methods help us easily access tuple elements.

Q.2. Slicing a tuple to access ranges of elements.

Ans: Tuple slicing is used to **get a part of a tuple** by selecting a range of elements. It works similar to list slicing.

The basic syntax is:

```
tuple[start:end]
```

Here **start** is the starting index and **end** is the ending index (not included).

Example

```
numbers = (10, 20, 30, 40, 50)

print(numbers[1:4])
```

Output:

```
(20, 30, 40)
```

This selects elements from index **1 to 3**.

Example without Start or End

```
print(numbers[:3])   # from beginning
print(numbers[2:])   # till end
```

Output:

```
(10, 20, 30)
(30, 40, 50)
```

Example with Step

```
print(numbers[0:5:2])
```

Output:

(10, 30, 50)

This selects elements by skipping every second element.

Conclusion

Tuple slicing helps us **access a range of elements easily** using start, end, and step values. It is useful when we need only a specific part of a tuple.

6. Dictionaries:

Q.1. Introduction to dictionaries: key-value pairs.

Ans: A **dictionary** in Python is a collection of data stored in **key-value pairs**. It is written using **curly brackets {}**. Each key is unique and is used to access its corresponding value.

Creating a Dictionary

Example:

```
student = {"name": "Mithlesh", "age": 25, "course": "Python"}
print(student)
```

Output:

{'name': 'Mithlesh', 'age': 25, 'course': 'Python'}

Here,

- **name, age, course** are keys
- **Mithlesh, 25, Python** are values

Accessing Values

We can access values using their keys.

Example:

```
print(student["age"])
```

Output:

25

Adding or Updating Values

We can add or update values using keys.

Example:

```
student["city"] = "Surat"
student["age"] = 26
print(student)
```

Conclusion

Dictionaries store data in **key-value pairs**, which makes it easy to organize and access data using keys instead of index positions.

Q.2. Accessing, adding, updating, and deleting dictionary elements.

Ans: In Python, we can easily work with dictionary elements using different operations like accessing, adding, updating, and deleting.

1. Accessing Elements

We can access values using their **keys**.

Example:

```
student = {"name": "Mithlesh", "age": 25}

print(student["name"])
```

Output:

```
Mithlesh
```

2. Adding Elements

We can add a new key-value pair by assigning a value to a new key.

Example:

```
student["city"] = "Surat"
print(student)
```

Output:

```
{'name': 'Mithlesh', 'age': 25, 'city': 'Surat'}
```

3. Updating Elements

We can update an existing value by using its key.

Example:

```
student["age"] = 26
print(student)
```

Output:

{'name': 'Mithlesh', 'age': 26, 'city': 'Surat'}

4. Deleting Elements

We can delete elements using del or pop().

Example:

```
del student["city"]
print(student)
```

Output:

{'name': 'Mithlesh', 'age': 26}

Another example:

```
student.pop("age")
print(student)
```

Conclusion

We can **access, add, update, and delete dictionary elements** using keys. These operations make dictionaries flexible and easy to manage in Python programs.

Q.3. Dictionary methods like keys(), values(), and items().

Ans: In Python, dictionaries provide built-in methods to access their data easily. The most commonly used methods are **keys()**, **values()**, and **items()**.

1. keys()

The **keys()** method returns all the **keys** in the dictionary.

Example:

```
student = {"name": "Mithlesh", "age": 25, "city": "Surat"}

print(student.keys())
```

Output:

dict_keys(['name', 'age', 'city'])

2. values()

The **values()** method returns all the **values** in the dictionary.

Example:

print(student.values())

Output:

dict_values(['Mithlesh', 25, 'Surat'])

3. items()

The **items()** method returns all **key-value pairs** as tuples.

Example:

print(student.items())

Output:

dict_items([('name', 'Mithlesh'), ('age', 25), ('city', 'Surat')])

Using with Loop

We can also use these methods with loops.

Example:

```
for key, value in student.items():
  print(key, value)
```

Conclusion

The methods **keys(), values(), and items()** help us easily access and work with dictionary data such as keys, values, and key-value pairs.

7. Working with Dictionaries:

Q.1. Iterating over a dictionary using loops.

Ans: Iterating over a dictionary means **going through its keys and values one by one**. We usually use a **for loop** for this.

1. Iterating Over Keys

By default, a loop goes through the **keys** of the dictionary.

Example:

```
student = {"name": "Mithlesh", "age": 25, "city": "Surat"}

for key in student:
  print(key)
```

Output:

name
age
city

2. Iterating Over Values

We can use the **values()** method to get all values.

Example:

```
for value in student.values():
  print(value)
```

Output:

Mithlesh
25
Surat

3. Iterating Over Key-Value Pairs

We can use the **items()** method to get both keys and values together.

Example:

```
for key, value in student.items():
  print(key, value)
```

Output:

name Mithlesh
age 25
city Surat

Conclusion

We can iterate over a dictionary using loops to access **keys, values, or both key-value pairs**. This helps us process dictionary data easily.

Q.2. Merging two lists into a dictionary using loops or zip().

Ans: We can combine two lists (one for keys and one for values) to create a **dictionary**. This can be done using a loop or the zip() function.

1. Using a Loop

We can use a **for loop** to create a dictionary manually.

Example:

```
keys = ["name", "age", "city"]
values = ["Mithlesh", 25, "Surat"]

result = {}

for i in range(len(keys)):
   result[keys[i]] = values[i]

print(result)
```

Output:

```
{'name': 'Mithlesh', 'age': 25, 'city': 'Surat'}
```

2. Using zip()

The **zip()** function combines elements from both lists and makes the process easier.

Example:

```
keys = ["name", "age", "city"]
values = ["Mithlesh", 25, "Surat"]

result = dict(zip(keys, values))

print(result)
```

Output:

```
{'name': 'Mithlesh', 'age': 25, 'city': 'Surat'}
```

Conclusion

We can merge two lists into a dictionary using a **loop** or the **zip() function**. The zip() method is shorter and more efficient, while loops give more control over the process.

Q.3. Counting occurrences of characters in a string using dictionaries.

Ans: We can use a **dictionary** to count how many times each character appears in a string. The character will be the **key**, and its count will be the **value**.

Example Using Loop

```
text = "python"

count = {}
```

```
for ch in text:
  if ch in count:
    count[ch] += 1
  else:
    count[ch] = 1

print(count)
```

Output:

{'p': 1, 'y': 1, 't': 1, 'h': 1, 'o': 1, 'n': 1}

Example with Repeated Characters

```
text = "banana"

count = {}

for ch in text:
  if ch in count:
    count[ch] += 1
  else:
    count[ch] = 1

print(count)
```

Output:

{'b': 1, 'a': 3, 'n': 2}

Conclusion

We use a dictionary to **store each character and its count**. By looping through the string, we can easily count how many times each character appears.

8. Functions:

Q.1. Defining functions in Python.

Ans: A **function** in Python is a block of code that performs a specific task. Functions help us **reuse code** and make programs more organized.

Syntax of a Function

```
def function_name():
  # code
```

We use the **def keyword** to define a function.

Example

```
def greet():
  print("Hello, welcome to Python")

greet()
```

Output:

Hello, welcome to Python

Here, greet() is the function, and it runs when we call it.

Function with Parameters

We can pass values to a function using **parameters**.

Example:

```
def add(a, b):
  print(a + b)

add(5, 3)
```

Output:

8

Function with Return Value

A function can also return a value using **return**.

Example:

```
def multiply(a, b):
  return a * b

result = multiply(4, 2)
print(result)
```

Output:

8

Conclusion

Functions are used to **organize code into reusable blocks**. We can define functions using def, pass parameters, and return values to make programs more efficient.

Q.2. Different types of functions: with/without parameters, with/without return values.

Ans: In Python, functions can be classified based on **parameters** and **return values**.

1. Function Without Parameters and Without Return Value

This type of function **does not take input** and **does not return anything**.

Example:

```
def greet():
  print("Hello")

greet()
```

Output:

Hello

2. Function With Parameters and Without Return Value

This function **takes input (parameters)** but **does not return a value**.

Example:

```
def add(a, b):
  print(a + b)

add(5, 3)
```

Output:

8

3. Function Without Parameters and With Return Value

This function **does not take input** but **returns a value**.

Example:

```
def get_number():
  return 10

print(get_number())
```

Output:

4. Function With Parameters and With Return Value

This function **takes input** and also **returns a value**.

Example:

```
def multiply(a, b):
  return a * b

result = multiply(4, 2)
print(result)
```

Output:

8

Conclusion

Functions in Python can be of different types based on **parameters and return values**. This helps us write flexible and reusable programs.

Q.3. Anonymous functions (lambda functions).

Ans: An **anonymous function** in Python is a function **without a name**. It is created using the **lambda keyword** and is usually used for short and simple operations.

Syntax

lambda arguments: expression

Example

```
add = lambda a, b: a + b
print(add(5, 3))
```

Output:

8

Here, the lambda function adds two numbers.

Example without Assigning to Variable

```
print((lambda x: x * x)(4))
```

Output:

16

Using Lambda with Functions

Lambda functions are often used with functions like map().

Example:

```
numbers = [1, 2, 3]
result = list(map(lambda x: x * 2, numbers))
print(result)
```

Output:

[2, 4, 6]

Conclusion

Lambda functions are **small, one-line functions** used for simple tasks. They make the code shorter and are useful when we do not need to define a full function using def.

9. Modules:

Q.1. Introduction to Python modules and importing modules.

Ans: A **module** in Python is a file that contains **functions, variables, or code** that we can reuse in other programs. Modules help us organize code and avoid rewriting the same logic again.

What is a Module?

For example, Python has built-in modules like **math** that provide useful functions.

Importing a Module

We can use a module by importing it using the **import keyword**.

Example:

```
import math

print(math.sqrt(16))
```

Output:

```
4.0
```

Import Specific Function

We can also import only required functions from a module.

Example:

```
from math import sqrt

print(sqrt(25))
```

Output:

```
5.0
```

Using Alias

We can give a module a short name using **alias**.

Example:

```
import math as m

print(m.pi)
```

Output:

```
3.141592653589793
```

Conclusion

Modules help us **reuse code and organize programs**. We can import them using import, from, or alias to make coding easier and more efficient.

Q.2. Standard library modules: math, random.

Ans: Python provides many built-in modules in its **standard library**. Two commonly used modules are **math** and **random**.

1. math Module

The **math module** provides functions for mathematical operations like square root, power, and constants.

Example:

```
import math

print(math.sqrt(16))   # square root
print(math.pow(2, 3))  # power
print(math.pi)         # value of pi
```

Output:

```
4.0
8.0
3.141592653589793
```

2. random Module

The **random module** is used to generate random numbers or select random items.

Example:

```
import random

print(random.randint(1, 10))   # random number between 1 and 10
```

Output (example):

```
7
```

Another example:

```
import random

fruits = ["apple", "banana", "mango"]
print(random.choice(fruits))
```

Output (example):

```
banana
```

Conclusion

The **math module** is used for mathematical calculations, while the **random module** is used to generate random values. Both are part of Python's standard library and are very useful in programs.

Q.3. Creating custom modules.

Ans: A **custom module** is a Python file (.py) that contains functions or variables which we can reuse in other programs. This helps us organize code and avoid repetition.

Step 1: Create a Module File

We create a Python file, for example:

mymodule.py

Inside this file, we define functions.

Example:

```
def greet(name):
  return "Hello " + name

def add(a, b):
  return a + b
```

Step 2: Import the Module

Now we use this module in another Python file using the **import** statement.

Example:

```
import mymodule

print(mymodule.greet("Mithlesh"))
print(mymodule.add(5, 3))
```

Output:

```
Hello Mithlesh
8
```

Step 3: Import Specific Functions

We can also import only required functions.

Example:

```
from mymodule import greet

print(greet("Mithlesh"))
```

Conclusion

We can create custom modules by writing code in a .py file and importing it into other programs. This helps us **reuse code, organize projects, and make programs cleaner**.